2.5. Классы и объекты. Наследование, инкапсуляция и полиморфизм. Создание класса обработки уязвимостей

2. Файлы, данные, простая обработка и визуализация

В этом уроке

- Что такое класс
- Как создать свой класс
- Какие у классов есть свойства

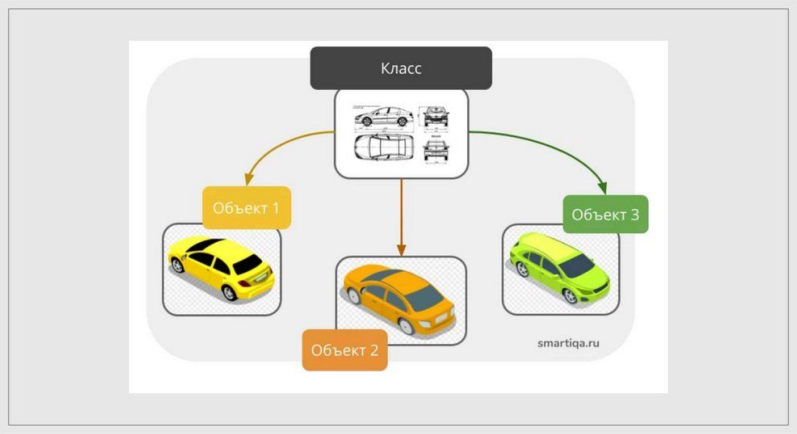
Объектно-ориентированное программирование

Объектно-ориентированное программирование (ООП) - методология программирования, основанная на представлении программы в виде совокупности объектов, каждый из которых является экземпляром определенного класса, а классы образуют иерархию наследования.

Python - объектно-ориентированный язык, а соответственно все в нем является объектом какого либо класса.

ООП. Термины

Класс = шаблон, который описывает свойства и возможные действия. Например, класс персонажей.



Объект класса = персонаж, построенный по шаблону класса.

Метод = функция, описанная внутри класса.

Инкапсуляция / наследование / полиморфизм

ООП. Создание класса

```
class ClassName:
    <statement-1>
    .
    .
    .
    <statement-N>
```

ООП. Создание класса

При создании экземпляра класса, как правило, требуется проводить его инициализацию (например, устанавливать начальные значения полей), для чего в Python предназначен специальный метод __init__

В данный метод мы можем передать необходимые значения и сохранить их в локальные переменные, значения которых будут разные для разных объектов.

Атрибуты по умолчанию

```
>>> def get_name(name='Иван'):
>>>     print(name)

>>> get_name()
>>> get_name('Петр')
```

ООП. Атрибуты по умолчанию

```
>>> class Car:
>>>    def __init __(self, speed, color='Yellow'):
>>>        self.speed = speed
>>>        self.color = color

>>> car1 = Car(100)
>>> car2 = Car(90, 'Blue')

>>> print(f'{car1.color=}')
>>> print(f'{car2.color=}')
```

ООП. Инкапсуляция

Инкапсуляция — ограничение доступа к составляющим объект компонентам (методам и переменным). Инкапсуляция делает некоторые из компонент доступными только внутри класса.

Повтор. Создание класса

```
class ClassName:
    <statement-1>
    .
    .
    .
    <statement-N>
```

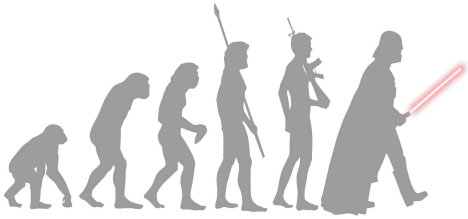
При создании экземпляра класса, как правило, требуется проводить его инициализацию (например, устанавливать начальные значения полей), для чего в Python предназначен специальный метод __init__

ООП. Наследование класса

```
class Transport:
    def retro():
        return 12

class Car(Transport):
    def retro():
        return 15
```

Когда один класс становится наследником другого, то все атрибуты класса-предка (надкласса, superclass) становятся доступны классу-потомку (подклассу, subclass) — наследуются (достаются в наследство).



ООП. Все будет super()

Функция super() предназначена для переопределения функции с сохранением связи с функцией родительского класса (доработка функции).

Вызов super заменяет обращение к self.

```
>>> class Bus(Transport):
>>>     def __init __(self, speed, color, seeds):
>>>         super().__init__(speed, color)
```

Перегрузка операторов

Перегрузка операторов — один из способов реализации полиморфизма, когда мы можем задать свою реализацию какого-либо метода в своём классе.

Полиморфизм – это способность одного и того же объекта вести себя по-разному в зависимости от того, в контексте какого класса он используется.

Перегрузка операторов

Список методов с двойным подчеркиванием:

```
__new__(cls[, …])
__init__(self[, ...])
__del__(self)
__repr__(self)
__str__(self)
__format__(self, format_spec)
__lt__(self, other)    x<y
__le__(self, other)    x <= y
__eq__(self, other)
__ne__(self, other)
__gt__(self, other)
__ge__(self, other)
__hash__(self)
```

__bool__(self)
__getattr__(self, name)
__setattr__(self, name, value)
__delattr__(self, name)
__call__(self[, args...])
__len__(self)
__getitem__(self, key)
__setitem__(self, key, value)
__delitem__(self, key)
__iter__(self)
__reversed__(self)
__contains__(self, item)

Перегрузка операторов

Таблица для перегрузки математических операторов
__add__(self, other) - сложение. x + y.
__sub__(self, other) - вычитание (x - y).
__mul__(self, other) - умножение (x * y).
__truediv__(self, other) - деление (x / y).
__floordiv__(self, other) - целочисленное деление (x // y).
__mod__(self, other) - остаток от деления (x % y).
__divmod__(self, other) - частное и остаток (divmod(x, y)).
__pow__(self, other[, modulo]) - возведение в степень (x ** y, pow(x, y[, modulo])).
__iadd__(self, other) - +=.
__isub__(self, other) - -=.
__imul__(self, other) - *=.
__itruediv__(self, other) - /=.
__ifloordiv__(self, other) - //=.
__imod__(self, other) - %=.
__ipow__(self, other[, modulo]) - **=.

Итоги урока

- Узнали про классы и как их создать
- Поняли что такое методы
- Разобрались со свойствами классов